1. Binary Tree Level Order (LC 102)

Problem: Return level-order traversal (BFS, left to right per level)

```java
// Time: O(n) - visit each node once
// Space: O(w) - queue width, O(n) worst case
public List<List<Integer>> levelOrder(TreeNode root) {
    List<List<Integer>> result = new ArrayList<>();
    if (root == null) return result; // O(1)

    Queue<TreeNode> queue = new LinkedList<>(); // O(w) space
    queue.offer(root); // O(1)

    while (!queue.isEmpty()) { // O(n) total iterations
        int size = queue.size(); // KEY: freeze level size
        List<Integer> level = new ArrayList<>(); // O(w) space

        for (int i = 0; i < size; i++) { // O(size)
            TreeNode node = queue.poll(); // O(1)
            level.add(node.val); // O(1) amortized

            if (node.left != null) queue.offer(node.left); // O(1)
            if (node.right != null) queue.offer(node.right);
        }
        result.add(level); // O(1)
    }
    return result;
}
```

Key Pattern: `int size = queue.size()` freezes current level size. Without it, newly added children would be processed in the same iteration!

Common Mistakes:

- Forgetting null check for root
- Not freezing size processes children same level
- Using `queue.poll()` without checking empty

Mini Dry Run: Tree: `[3,9,20,15,7]`

```
Level 0: queue=[3], process 3   [[3]]
Level 1: queue=[9,20], process both   [[3],[9,20]]
Level 2: queue=[15,7]   [[3],[9,20],[15,7]]
```

Complexity:

- **Time:** O(n) - each node visited once
- **Space:** O(w) for queue where w = max width
- Complete tree: w = n/2 O(n)
- Skewed tree: w = 1 O(1)

2. Zigzag Level Order (LC 103)

Problem: Alternate left-to-right and right-to-left per level

```java
// Time: O(n) - visit + reverse
// Space: O(w) for queue
public List<List<Integer>> zigzagLevelOrder(TreeNode root) {
    List<List<Integer>> result = new ArrayList<>();
    if (root == null) return result;

    Queue<TreeNode> queue = new LinkedList<>();
    queue.offer(root);
    boolean leftToRight = true; // Track direction

    while (!queue.isEmpty()) {
        int size = queue.size();
        List<Integer> level = new ArrayList<>();

        for (int i = 0; i < size; i++) {
            TreeNode node = queue.poll();
            level.add(node.val);
            if (node.left != null) queue.offer(node.left);
            if (node.right != null) queue.offer(node.right);
        }

        if (!leftToRight) {
            Collections.reverse(level); // O(level_size)
        }
        result.add(level);
        leftToRight = !leftToRight; // Toggle
    }
    return result;
}
```

Optimization: Can use `LinkedList` with `addFirst/addLast` for O(1) insertion, but `Collections.reverse()` is cleaner and same complexity overall.

Alternative DFS (Not Recommended):

```java
// Time: O(n^2) worst case with ArrayList.add(0,val)
// Use LinkedList if going this route!
private void dfs(TreeNode node, int level, List<List<Integer>> result) {
    if (node == null) return;
    if (level == result.size()) result.add(new LinkedList<>());

    LinkedList<Integer> levelList = (LinkedList) result.get(level);
    if (level % 2 == 0) levelList.addLast(node.val); // O(1)
    else levelList.addFirst(node.val); // O(1)

    dfs(node.left, level + 1, result);
    dfs(node.right, level + 1, result);
}
```

DFS Mistake: Using `ArrayList.add(0, val)` is O(n) per insertion due to shifting! Use `LinkedList` for O(1) addFirst.

3. Minimum Depth (LC 111)

Problem: Shortest path from root to any leaf

```java
// Time: O(n) worst case, better average
// Space: O(w) for queue
public int minDepth(TreeNode root) {
    if (root == null) return 0;

    Queue<TreeNode> queue = new LinkedList<>();
    queue.offer(root);
    int depth = 0;

    while (!queue.isEmpty()) {
        depth++; // Increment per level
        int size = queue.size();

        for (int i = 0; i < size; i++) {
            TreeNode node = queue.poll();

            // KEY: Early termination at first leaf
            if (node.left == null && node.right == null) {
                return depth; // O(1) - found answer!
            }

            if (node.left != null) queue.offer(node.left);
            if (node.right != null) queue.offer(node.right);
        }
    }
    return depth;
}
```

Why BFS > DFS: BFS finds the first (shallowest) leaf immediately. DFS must explore all paths to find minimum.

DFS Alternative (Less Efficient):

```java
// Time: O(n) - must visit all nodes
// Space: O(h) - recursion stack
public int minDepth(TreeNode root) {
    if (root == null) return 0;
    if (root.left == null && root.right == null) return 1;

    // Handle one-sided trees carefully!
    int left = (root.left != null) ? minDepth(root.left) : Integer.MAX_VALUE;
    int right = (root.right != null) ? minDepth(root.right) : Integer.MAX_VALUE;

    return 1 + Math.min(left, right);
}
```

DFS Mistake: Can't use simple `1 + min(left, right)` because null child returns 0, but that's not a valid path to leaf!

4. Right Side View (LC 199)

Problem: Return values of nodes visible from right side

```java
// Time: O(n) - visit each node
// Space: O(w) for queue
public List<Integer> rightSideView(TreeNode root) {
    List<Integer> result = new ArrayList<>();
    if (root == null) return result;

    Queue<TreeNode> queue = new LinkedList<>();
    queue.offer(root);

    while (!queue.isEmpty()) {
        int size = queue.size();

        for (int i = 0; i < size; i++) {
            TreeNode node = queue.poll();

            // Last node of level is rightmost visible
            if (i == size - 1) { // O(1) check
                result.add(node.val);
            }

            if (node.left != null) queue.offer(node.left);
            if (node.right != null) queue.offer(node.right);
        }
    }
    return result;
}
```

Elegant Alternative:

```java
// Keep updating node, last one is rightmost
while (!queue.isEmpty()) {
    int size = queue.size();
    TreeNode node = null; // Declare here
    for (int i = 0; i < size; i++) {
        node = queue.poll(); // Keep updating
        if (node.left != null) queue.offer(node.left);
        if (node.right != null) queue.offer(node.right);
    }
    result.add(node.val); // Add after loop
}
```

DFS Alternative: Visit right before left, add when first reaching each level. But BFS is more intuitive for "view" problems.

5. Same Tree (LC 100) - DFS

Problem: Check if two trees are identical

```java
// Time: O(min(n,m)) - stop at first difference
// Space: O(h) - recursion stack
public boolean isSameTree(TreeNode p, TreeNode q) {
    // Base case: both null
    if (p == null && q == null) return true; // O(1)

    // One null, other not
    if (p == null || q == null) return false; // O(1)

    // Different values
    if (p.val != q.val) return false; // O(1)

    // Recursively check subtrees
    return isSameTree(p.left, q.left) && // O(left_size)
           isSameTree(p.right, q.right); // O(right_size)
}
```

Call Stack Example: Trees: `[1,2,3]` vs `[1,2,3]`

```
Call 1: isSameTree(1,1)  vals match
  Call 2: isSameTree(2,2)  vals match
    Call 3: isSameTree(null,null)  true
    Call 4: isSameTree(null,null)  true
  Return: true && true = true
  Call 5: isSameTree(3,3)  vals match
    Call 6: isSameTree(null,null)  true
    Call 7: isSameTree(null,null)  true
  Return: true && true = true
Return: true && true = TRUE
```

Common Mistake: Not checking null cases separately leads to NullPointerException when accessing `p.val`.

6. Symmetric Tree (LC 101) - DFS

Problem: Check if tree is mirror of itself

```java
// Time: O(n) - visit each node once
// Space: O(h) - recursion stack
public boolean isSymmetric(TreeNode root) {
    if (root == null) return true;
    return isMirror(root.left, root.right);
}

private boolean isMirror(TreeNode left, TreeNode right) {
    // Both null - symmetric
    if (left == null && right == null) return true; // O(1)

    // One null - not symmetric
    if (left == null || right == null) return false; // O(1)

    // Check: values match AND outer edges match AND inner edges match
    return (left.val == right.val) && // O(1)
           isMirror(left.left, right.right) && // Outer
           isMirror(left.right, right.left); // Inner - KEY!
}
```

Key Insight: Compare `left.left` with `right.right` (outer) and `left.right` with `right.left` (inner). This is the mirror property!

BFS Alternative:

```java
// Process pairs simultaneously
Queue<TreeNode> queue = new LinkedList<>();
queue.offer(root.left);
queue.offer(root.right);

while (!queue.isEmpty()) {
    TreeNode t1 = queue.poll();
    TreeNode t2 = queue.poll();

    if (t1 == null && t2 == null) continue; // KEY: continue, not return!
    if (t1 == null || t2 == null) return false;
    if (t1.val != t2.val) return false;

    queue.offer(t1.left); queue.offer(t2.right); // Outer
    queue.offer(t1.right); queue.offer(t2.left); // Inner
}
return true;
```

BFS Mistake: Using `return true` when both null would exit prematurely. Must use `continue` to check remaining pairs!

7. Maximum Depth (LC 104) - DFS

Problem: Find height of tree (longest path to leaf)

```java
// Time: O(n) - visit all nodes
// Space: O(h) - recursion stack
public int maxDepth(TreeNode root) {
    if (root == null) return 0; // Base case: O(1)

    // Post-order: process children first
    int left = maxDepth(root.left); // O(left_subtree)
    int right = maxDepth(root.right); // O(right_subtree)

    return 1 + Math.max(left, right); // O(1)
}
```

Call Stack: Tree: `[3,9,20,15,7]`

```
maxDepth(3)
 maxDepth(9)  1
 maxDepth(20)
    maxDepth(15)  1
    maxDepth(7)  1
   Return: 1 + max(1,1) = 2
Return: 1 + max(1,2) = 3
```

Why Post-Order: Must know children's depths before computing current node's depth. This is bottom-up computation.

BFS Alternative:

```java
// Count levels
int depth = 0;
while (!queue.isEmpty()) {
    depth++; // O(1)
    int size = queue.size();
    for (int i = 0; i < size; i++) {
        // Process level...
    }
}
return depth;
```

8. Path Sum (LC 112) - DFS

Problem: Check if root-to-leaf path sums to target

```java
// Time: O(n) worst case, better with early exit
// Space: O(h) - recursion stack
public boolean hasPathSum(TreeNode root, int targetSum) {
    if (root == null) return false; // O(1)

    // Leaf check - KEY: must be leaf!
    if (root.left == null && root.right == null) {
        return root.val == targetSum; // O(1)
    }

    // Subtract current value, check subtrees
    return hasPathSum(root.left, targetSum - root.val) || // Short-circuit!
           hasPathSum(root.right, targetSum - root.val);
}
```

Top-Down Subtraction > Bottom-Up Addition:

- Subtraction: 2 parameters (node, remaining)
- Addition: 3 parameters (node, target, currentSum)
- Subtraction is cleaner and more idiomatic

Bottom-Up Alternative:

```java
// Less preferred - extra parameter
private boolean dfs(TreeNode node, int target, int sum) {
    if (node == null) return false;
    sum += node.val;
    if (node.left == null && node.right == null) {
        return sum == target;
    }
    return dfs(node.left, target, sum) ||
           dfs(node.right, target, sum);
}
```

Common Mistake: Checking `targetSum == 0` at null nodes instead of at leaves. Must reach an actual leaf node!

9. Lowest Common Ancestor (LC 236)

Problem: Find LCA of two nodes in binary tree

```java
// Time: O(n) - visit each node once
// Space: O(h) - recursion stack
public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
    // Base cases: null or found p or q
    if (root == null || root == p || root == q) {
        return root; // O(1) - immediate return
    }

    // Search in subtrees
    TreeNode left = lowestCommonAncestor(root.left, p, q); // O(left)
    TreeNode right = lowestCommonAncestor(root.right, p, q); // O(right)

    // If found in both subtrees, current node is LCA
    if (left != null && right != null) {
        return root; // O(1) - found LCA!
    }
```

```
    // Return whichever side found something
    return left != null ? left : right; // O(1)
}
```

Key Cases:

1. **Different subtrees:** left = p, right = q return root
2. **One is ancestor:** When we hit p, return immediately, never recurse deeper
3. **Same subtree:** One side returns LCA, other returns null

Call Stack: Tree: `[3,5,1,6,2,0,8]`, p=6, q=8

```
LCA(3)
 LCA(5)  searches left
   LCA(6)  returns 6 (found p!)
   LCA(2)  returns null
  Returns: 6 (found p in left subtree)
 LCA(1)  searches right
    LCA(0)  returns null
    LCA(8)  returns 8 (found q!)
   Returns: 8 (found q in right subtree)
Returns: root (3)  left=6, right=8, both non-null
```

BST Optimization: If BST, use values to decide which subtree:

```
if (p.val < root.val && q.val < root.val)
    return LCA(root.left, p, q); // Both in left
if (p.val > root.val && q.val > root.val)
    return LCA(root.right, p, q); // Both in right
return root; // Split point or one is root
```

Time improves to O(h) by pruning branches!

10. Binary Tree Pruning (LC 814)

Problem: Remove subtrees that don't contain 1

```
// Time: O(n) - visit each node once
// Space: O(h) - recursion stack
public TreeNode pruneTree(TreeNode root) {
    if (root == null) return null; // O(1)

    // Post-order: prune children first
    root.left = pruneTree(root.left); // O(left) - may return null
    root.right = pruneTree(root.right); // O(right) - may return null

    // After pruning children, check if current should be removed
    // Prune if: value is 0 AND both children are null (leaf)
    if (root.val == 0 && root.left == null && root.right == null) {
        return null; // O(1) - prune this node
    }

    return root; // O(1) - keep this node
}
```

Why Post-Order Critical: Must prune children first to know if current node becomes a leaf after pruning!

Example: `[1,0,1,0,0,0,1]`

```
Original:     1          After pruning:  1
            / \                          \
           0   1                          1
          / \ / \                          \
         0  0 0  1                          1
```

```
Process order (post-order):
1. Prune leaf 0s  parent 0 becomes leaf
2. Prune parent 0  root.left = null
3. Keep right subtree (has 1)
```

Pre-order Mistake: Checking before recursing won't work - children might become prune-able after their descendants are removed!

11. Delete Leaf Nodes (LC 1325)

Problem: Remove all leaf nodes with given value (may create new leaves to remove)

```
// Time: O(n) - visit each node
// Space: O(h) - recursion stack
public TreeNode removeLeafNodes(TreeNode root, int target) {
    if (root == null) return null; // O(1)

    // Post-order: process children first
    root.left = removeLeafNodes(root.left, target); // O(left)
    root.right = removeLeafNodes(root.right, target); // O(right)

    // After pruning children, check if current is leaf with target
    if (root.val == target &&
        root.left == null &&
        root.right == null) {
        return null; // O(1) - remove this leaf
    }

    return root; // O(1) - keep
}
```

N-ary Tree Version:

```
// Time: O(n), Space: O(h + k) where k = max children
public Node removeLeafNodes(Node root, int target) {
    if (root == null) return null;

    // Build new children list without pruned nodes
    List<Node> newChildren = new ArrayList<>(); // O(k) space
    for (Node child : root.children) { // O(k) iterations
        Node result = removeLeafNodes(child, target); // O(subtree)
        if (result != null) { // O(1) check
            newChildren.add(result); // O(1) amortized
        }
    }
    root.children = newChildren; // O(1) assignment

    // Check if current node should be pruned
    if (root.val == target && root.children.isEmpty()) {
        return null; // O(1)
    }

    return root;
}
```

Cascading Effect: Removing leaf nodes can create new leaf nodes with target value. Post-order handles this automatically by processing bottom-up.

12. Flood Fill (LC 733) - Grid DFS

Problem: Change connected pixels of same color to new color

```
// Time: O(m*n) - visit each cell once
// Space: O(m*n) worst case - recursion stack
public int[][] floodFill(int[][] image, int sr, int sc, int color) {
    int original = image[sr][sc]; // O(1)

    // Optimization: skip if already correct color
    if (original != color) { // O(1) check
        dfs(image, sr, sc, color, original);
    }

    return image;
}

private void dfs(int[][] image, int sr, int sc, int color, int original) {
    // Boundary checks + color checks
    if (sr < 0 || sr >= image.length || // O(1) each
        sc < 0 || sc >= image[0].length ||
        image[sr][sc] == color || // Already painted (prevents infinite loop!)
        image[sr][sc] != original) { // Different color (boundary)
        return; // O(1)
    }

    image[sr][sc] = color; // O(1) - paint & mark visited

    // Explore 4 directions
    dfs(image, sr - 1, sc, color, original); // Up: O(cells_above)
    dfs(image, sr + 1, sc, color, original); // Down: O(cells_below)
    dfs(image, sr, sc - 1, color, original); // Left: O(cells_left)
    dfs(image, sr, sc + 1, color, original); // Right: O(cells_right)
}
```

In-Place Visited Marking: Setting `image[sr][sc] = color` acts as visited marker. No need for separate `boolean[][] visited` array!

Mini Execution: `[[1,1,1],[1,1,0],[1,0,1]]`, sr=1, sc=1, color=2

```
Original:     Paint Order:     Final:
1 1 1         1  2  6          2 2 2
1 1*0                          2 2 0
1 0 1         3  4             2 0 1
              5 (stops at 0)
```

```
Stack depth: 6 (worst path)
Cells painted: 6 (all connected 1s)
```

Infinite Loop Risk: Without `image[sr][sc] == color` check, cell would call neighbor, neighbor calls back stack overflow!

BFS Alternative:

```
// Same time, more space-explicit
Queue<int[]> queue = new LinkedList<>();
queue.offer(new int[]{sr, sc});
image[sr][sc] = color;

int[][] dirs = {{-1,0}, {1,0}, {0,-1}, {0,1}};
while (!queue.isEmpty()) {
    int[] cell = queue.poll();
    for (int[] dir : dirs) {
        int nr = cell[0] + dir[0], nc = cell[1] + dir[1];
        if (valid && image[nr][nc] == original) {
            image[nr][nc] = color;
            queue.offer(new int[]{nr, nc});
```

```
    }
  }
}
```

13. Rotting Oranges (LC 994) - Multi-Source BFS

Problem: Find minimum minutes for all oranges to rot (or -1 if impossible)

```
// Time: O(m*n) - visit each cell once
// Space: O(m*n) - queue can hold all cells
public int orangesRotting(int[][] grid) {
    int rows = grid.length, cols = grid[0].length; // O(1)
    Queue<int[]> queue = new LinkedList<>(); // O(m*n) space worst case
    int freshCount = 0;

    // Phase 1: Find all initial rotten oranges & count fresh
    for (int i = 0; i < rows; i++) { // O(m*n) total
        for (int j = 0; j < cols; j++) {
            if (grid[i][j] == 2) {
                queue.offer(new int[]{i, j}); // O(1) - multi-source!
            } else if (grid[i][j] == 1) {
                freshCount++; // O(1)
            }
        }
    }

    if (freshCount == 0) return 0; // O(1) - no work needed

    // Phase 2: Multi-source BFS
    int minutes = 0;
    int[][] dirs = {{-1,0}, {1,0}, {0,-1}, {0,1}};

    while (!queue.isEmpty()) { // O(m*n) total iterations
        int size = queue.size(); // KEY: freeze level
        boolean rotted = false; // Track if anything rotted this minute

        for (int i = 0; i < size; i++) { // O(size)
            int[] curr = queue.poll(); // O(1)
            int row = curr[0], col = curr[1];

            // Check 4 neighbors
            for (int[] dir : dirs) { // O(4) = O(1)
                int nr = row + dir[0], nc = col + dir[1];

                // If valid and fresh, rot it
                if (nr >= 0 && nr < rows && nc >= 0 && nc < cols &&
                    grid[nr][nc] == 1) {
                    grid[nr][nc] = 2; // O(1) - rot & mark visited
                    queue.offer(new int[]{nr, nc}); // O(1)
                    freshCount--; // O(1)
                    rotted = true; // O(1)
                }
            }
        }

        if (rotted) minutes++; // O(1) - only increment if we rotted something
    }

    return freshCount == 0 ? minutes : -1; // O(1)
}
```

Multi-Source BFS Pattern:
1. Add ALL source points to queue initially
2. Process level by level (each level = 1 time unit)
3. Use `boolean rotted` to track if work was done
4. Check remaining fresh count at end

This pattern applies to: Walls & Gates, 01 Matrix, As Far from Land

Mini Execution: `[[2,1,1],[1,1,0],[0,1,1]]`

```
Min 0:          Min 1:          Min 2:          Min 3:          Min 4:
2 1 1           2 2 1           2 2 2           2 2 2
1 1 0    ->     2 1 0    ->     2 2 0    ->     2 2 0
0 1 1           0 1 1           0 1 1           0 2 1

Queue size:   [(1,0),(0,1)] [(1,1),(0,2)] [(2,1)]
Fresh: 64     42            21            10
```

Why O(m*n) not O((m*n)š)?
- Each cell enters queue at most once
- Each cell is processed (popped) at most once
- Total queue operations: O(m*n)
- Not O((m*n)š) because we don't revisit!

Common Mistakes:
- Incrementing minutes every loop (should only if `rotted`)
- Not using `size = queue.size()` processes children same minute
- Forgetting initial `freshCount == 0` check

Key Patterns Summary

BFS Template

```
Queue<Node> queue = new LinkedList<>();
queue.offer(start);
Set<Node> visited = new HashSet<>(); // Or mark in-place
visited.add(start);

while (!queue.isEmpty()) {
    int size = queue.size(); // Level-by-level
    for (int i = 0; i < size; i++) {
        Node node = queue.poll();
        // Process node
        for (Node neighbor : getNeighbors(node)) {
            if (!visited.contains(neighbor)) {
                visited.add(neighbor);
                queue.offer(neighbor);
            }
        }
    }
}
```

DFS Template (Recursion)

```
Set<Node> visited = new HashSet<>();

void dfs(Node node) {
    if (node == null || visited.contains(node)) return;
    visited.add(node);
    // Process node
    for (Node neighbor : getNeighbors(node)) {
        dfs(neighbor);
    }
}
```

Grid DFS Template

```
void dfs(int[][] grid, int r, int c) {
    // Boundary & visited checks
    if (r < 0 || r >= m || c < 0 || c >= n ||
        grid[r][c] == VISITED) return;

    grid[r][c] = VISITED; // Mark in-place

    // 4 directions
    dfs(grid, r-1, c); dfs(grid, r+1, c);
    dfs(grid, r, c-1); dfs(grid, r, c+1);
}
```

Post-Order Tree Template

```
Result postOrder(TreeNode root) {
    if (root == null) return baseCase;

    Result left = postOrder(root.left); // Process left
    Result right = postOrder(root.right); // Process right

    return combine(root, left, right); // Process current
}
```

When to Use What?

Problem Type	Use BFS	Use DFS
Shortest path (unweighted)		
Level-order traversal		
Minimum depth		
Maximum depth		
Path existence		
Tree modification		
Connected components		
Multi-source propagation		

Space Complexity Comparison

Tree Shape	BFS Queue	DFS Stack
Complete/Perfect	O(n)	O(log n)
Balanced	O(n)	O(log n)
Skewed (linked list)	O(1)	O(n)

Time Complexity Cheat Sheet

Tree Operations:
- Single traversal (BFS/DFS): O(n)
- With extra work per node: O(n Œ work)
- Post-order with combining: O(n)
- Level-order with reverse: O(n) - reverse is O(level), sum = O(n)

Grid Operations:

- DFS/BFS on grid: O(m Œ n)
- Multi-source BFS: O(m Œ n) - each cell visited once
- 4-directional: constant factor, still O(m Œ n)

Common Mistakes:

- `ArrayList.add(0, val)`: O(n) due to shifting!
- `Collections.reverse(list)`: O(list.size())
- `LinkedList.addFirst/addLast`: O(1)

Interview Pro Tips

Problem Recognition

See "level" or "shortest" BFS **See "path" or "all nodes"** DFS **See "connected" or "region"** DFS/BFS on grid **See "time" or "distance"** Multi-source BFS

Optimization Tricks

1. **Early termination:** Return immediately when answer found (min depth, path sum)
2. **In-place marking:** Modify grid/tree to mark visited (flood fill, pruning)
3. **Level freezing:** Use `size = queue.size()` for level-by-level
4. **Short-circuit:** Use `||` and `&&` for early exit
5. **Skip work:** Check if already done (flood fill same color check)

Debugging Checklist

- Null checks first (before accessing fields)
- Boundary checks for grids (0 r < m, 0 c < n)
- Visited marking (prevent infinite loops)
- Base cases in recursion (null, leaf, boundary)
- Post-order for bottom-up computation
- Level size freezing for BFS

Common Pitfalls

Top Mistakes:

1. Not checking null before accessing `node.val`
2. Boundary check AFTER array access ArrayIndexOutOfBounds
3. Using ArrayList.add(0) instead of LinkedList
4. Not marking visited infinite recursion/loop
5. Checking parent before pruning children (need post-order!)
6. Incrementing time when no work done (need `rotted` flag)

Must-Know Complexities

Data Structure Operations:

Operation	ArrayList	LinkedList
add(element)	O(1) amortized	O(1)
add(0, element)	O(n)	O(1)
get(index)	O(1)	O(n)
remove(index)	O(n)	O(n)

Queue Operations: offer/poll/peek all O(1)
Set Operations: add/contains/remove all O(1) average for HashSet

Quick Reference: Problem Patterns

Level-Order Patterns:

- Normal: Add all, return all levels
- Zigzag: Add all, reverse alternate levels
- Right view: Add only last of each level
- Min depth: Return depth at first leaf

Tree Comparison Patterns:

- Same tree: Compare same sides (left-left, right-right)
- Symmetric: Compare opposite sides (left-right, right-left)

Path Patterns:

- Path sum: Subtract going down, check at leaf
- All paths: Backtracking with path list
- Max/min path: Post-order, compare children

Modification Patterns:

- Pruning: Post-order, check after children pruned
- Invert: Pre-order, swap before recursing
- Flatten: In-order with tracking previous

Grid Patterns:

- Flood fill: DFS with in-place marking
- Islands: DFS/BFS, count components
- Multi-source: BFS from all sources simultaneously
- Distance: BFS level = distance

Final Exam Questions

Q1: When should you use `int size = queue.size()`? **A:** When processing level by level in BFS, to prevent newly added nodes from being processed in the same iteration.

Q2: Why is flood fill DFS better than BFS? **A:** Both work, but DFS is simpler code. BFS uses more memory with queue of coordinate pairs.

Q3: When do you need post-order traversal? **A:** When current node's decision depends on children's results (pruning, max depth, LCA).

Q4: How to prevent infinite loops in graphs? **A:** Mark visited (HashSet or in-place modification).

Q5: Why is multi-source BFS O(n) not O(nš)? **A:** Each cell is visited and processed at most once, regardless of number of sources.

Q6: When to use LinkedList over ArrayList? **A:** When frequently adding to front (addFirst) or need O(1) insertions at both ends.

Q7: How to handle "impossible" cases? **A:** Track count/flag during traversal, check at end (freshCount in rotting oranges).

Q8: Pre-order vs post-order for tree modification? **A:** Post-order for pruning (need to see children first), pre-order for copying/cloning.